

ICPC Code Template

喵喵喵幼儿园

6 CATEGORIES

10 TEMPLATES

442 SOURCE LINES

Quick Index

1 Data Structures	2
1.1 01 树状数组	2
1.2 $O(1)$ 查询树状数组	2
1.3 Top Tree	3
2 Graph / Flow	3
2.1 最大流	3
3 Math	4
3.1 Binary GCD	4
4 Misc	5
4.1 快速模乘	5
5 Poly	5
5.1 多项式求逆	5
5.2 多项式乘法	6
6 String	7
6.1 后缀数组	7
6.2 后缀树	8

1 Data Structures

1.1 01 树状数组

FILE data-structures/fwt_bitset.cpp

LINES 41

USE 下标从 1 开始，激活位置并查询区间内已激活位置数

COST Add/query $O(\log(N/64))$, in-block query $O(1)$

AUTHOR ppip

```

1 struct fwt {
2     static constexpr int B{(N>>6)+5};
3     int t[B],n;
4     unsigned long long b[B];
5     void clear() {
6         n=(n>>6)+1;
7         fill_n(t,n+1,0);
8         fill_n(b,n+1,0);
9     }
10    int px(int x) {
11        int y{x&63},z{x>>6};
12        return __builtin_popcountll(b[z]&(~0ull>>(63-y)));
13    }
14    int qry(int l,int r) {
15        --l;
16        int ans{px(r)-px(l)};
17        l>>=6;r>>=6;
18        while (r>l) ans+=t[r],r&=r-1;
19        while (l>r) ans-=t[l],l&=l-1;
20        return ans;
21    }
22    void add(int x) {
23        unsigned long long w{1ull<<(x&63)};
24        if (b[x>>6]&w) return;
25        b[x>>6]|=w;
26        x=(x>>6)+1;
27        while (x<=n) {
28            ++t[x];
29            x+=x&-x;
30        }
31    }
32    int qry(int x) {
33        int ans{px(x)};
34        x>>=6;
35        while (x) {
36            ans+=t[x];
37            x&=x-1;
38        }
39        return ans;
40    }
41 } t;

```

1.2 $O(1)$ 查询树状数组

FILE data-structures/fwt_suffix.cpp

LINES 16

USE 下标从 1 开始，单点加、查询后缀和；适合查询远多于修改

COST Query $O(1)$, add $O(128+N/4096)$

AUTHOR ppip

```

1 struct FWT {
2     int s1[N+5],s2[(N>>6)+5],s3[(N>>12)+5];
3     void init() {
4         memset(s1,0,sizeof(s1));
5         memset(s2,0,sizeof(s2));
6         memset(s3,0,sizeof(s3));
7     }
8     void add(int x,int y) {
9         for (int i{(x>>6)<<6};i<=x;++i) s1[i]+=y;
10        for (int i{(x>>12)<<6};i<(x>>6);++i) s2[i]+=y;
11        for (int i{0};i<(x>>12);++i) s3[i]+=y;
12    }
13    int qry(int x) {
14        return s1[x]+s2[x>>6]+s3[x>>12];
15    }
16 } T;

```

1.3 Top Tree

FILE data-structures/toptree.cpp

LINES 60

USE Magic tree that can fulfill every wish.

COST Tree height $O(\log n)$

AUTHOR ppip

```

1 vector<pair<int,int>> e[N+5];
2 int sz[N+5],fa[N+5],son[N+5];
3 void TOPTREE_INIT(int u) {
4     sz[u]=1;
5     for (auto [v, _]:e[u])
6         if (v!=fa[u]) {
7             TOPTREE_INIT(v);
8             sz[u]+=sz[v];
9             if (sz[v]>sz[son[u]]) son[u]=v;
10        }
11    if (son[u]) {
12        int si{exchange(sz[son[u]],0)};
13        sort(e[u].begin(),e[u].end(),[](auto x,auto y){return sz[x.first]<sz[y.first]});
14        sz[son[u]]=si;
15    }
16 }
17 struct cluster {
18     //....
19 } t[N*2+5];
20 int cnode;
21 using P=pair<int,int>;
22 using iter=vector<P>::iterator;
23 int merge(iter L,iter R,char tp) {
24     if (L+1==R) {
25         int z{L->first};
26         if (z<=n) t[z].u=fa[z],t[z].v=z;
27         return z;
28     }
29     iter p{L+1};
30     int sum{accumulate(p,R,-L->second,[](int x,P y){return x+y.second;})};
31     while (p+1!=R&&sum>p->second) {
32         sum-=p->second*2;
33         ++p;
34     }
35     int u{++cnode};
36     t[u].t=tp;
37     t[u].l=merge(L,p,tp);
38     t[u].r=merge(p,R,tp);
39     t[t[u].l].f=t[t[u].r].f=u;
40     t[u].u=t[t[u].l].u;
41     t[u].v=(tp=='C'?t[t[u].r].v:t[t[u].l].v);
42     return u;
43 }
44 int build(int u) {
45     vector<P> v1;v1.emplace_back(u,1);
46     while (son[u]) {
47         vector<P> v2;v2.emplace_back(son[u],1);
48         for (auto [v, _]:e[u])
49             if (v!=fa[u]&&v!=son[u]) v2.emplace_back(build(v),sz[v]);
50         v1.emplace_back(merge(v2.begin(),v2.end(),'R'),sz[u]-sz[son[u]]);
51         u=son[u];
52     }
53     return merge(v1.begin(),v1.end(),'C');
54 }
55 int main() {
56     cnode=n;
57     TOPTREE_INIT(1);
58     int rt{build(1)};
59     return 0;
60 }

```

2 Graph / Flow

2.1 最大流

FILE graph/flow/maxflow.cpp

LINES 60

USE Dinic, 要求汇点编号为最大值

COST 相信的心就是魔法

AUTHOR ppip

```

1 struct edge {
2     int u,v,w,nxt;
3     edge(int __,int __,int __,int __):
4         u{__},v{__},w{__},nxt{__} {}
5     edge() {}
6 };
7 vector<edge> e(2);
8 int hd[N+5];
9 void _add(int u,int v,int w) {
10     e.emplace_back(u,v,w,hd[u]);
11     hd[u]=e.size()-1;
12 }
13 void add(int u,int v,int w) {
14     _add(u,v,w);
15     _add(v,u,0);
16 }
17 int q[N+5];
18 int dep[N+5],cur[N+5];
19 bool bfs(int s,int t) {
20     dep[s]=1;
21     int l{0},r{0};
22     q[r++]=s;
23     while (r-l) {
24         int u{q[l++]};
25         for (int x{hd[u]};x;x=e[x].nxt)
26             if (!dep[e[x].v]&&e[x].w) {
27                 dep[e[x].v]=dep[u]+1;
28                 q[r++]=e[x].v;
29             }
30     }
31     return dep[t];
32 }
33 long long dfs(int s,int t,long long flow) {
34     if (s==t||!flow) return flow;
35     long long ans{0};
36     for (int &x{cur[s]},d;x;x=e[x].nxt) {
37         if (dep[e[x].v]==dep[s]+1&&e[x].w&&(d=dfs(e[x].v,t,min(flow,(long long)e[x].w)))) {
38             flow-=d;
39             ans+=d;
40             e[x].w-=d;
41             e[x^1].w+=d;
42             if (!flow) return ans;
43         }
44     }
45     return ans;
46 }
47 long long calc(int s,int t) {
48     fill_n(dep+1,t,0);
49     long long flow{0};
50     while (bfs(s,t)) {
51         copy(hd+1,hd+t+1,cur+1);
52         flow+=dfs(s,t,LLONG_MAX);
53         fill_n(dep+1,t,0);
54     }
55     return flow;
56 }
57 void clear(int t) {
58     e.resize(2);
59     fill_n(hd+1,t,0);
60 }

```

3 Math

3.1 Binary GCD

FILE math/binary_gcd.cpp

LINES 6

USE 超快 gcd 算法

COST $O(\log n)$, 可以当作 $O(1)$

AUTHOR ppip

```

1 #define ctz __builtin_ctz
2 inline int gcd(int a,int b) {
3     int az{ctz(a)},bz{ctz(b)},z{min(az,bz)},t;b>>=bz;
4     while (a) a>>=az,t=a-b,az=ctz(t),b=min(a,b),a=abs(t);
5     return b<<z;
6 }

```

4 Misc

4.1 快速模乘

FILE misc/fastmod.cpp

LINES 16

USE 要求 $1 \leq P < 2^{31}$ 且 $a < P$; 同一 z 多次使用时先 `trans`, 再用 `mul_t`COST $O(1)$

AUTHOR ppip

```

1 using u32=unsigned;
2 using u64=unsigned long long;
3 using u128=unsigned __int128;
4 const u32 P=998244353;
5 inline u64 trans(u64 x) {
6     constexpr u64 A=-(u64)P/P+1;
7     constexpr u64 q=((u128(-(u64)P)%P)<<64)+P-1)/P;
8     return x*A+u64((u128)x*q>>64)+1;
9 }
10 // t 必须是 trans(z) 的返回值; 复用同一 z 时只需计算一次 t
11 inline u32 mul_t(u32 a,u64 t) {
12     return a*t*(u128)P>>64;
13 }
14 inline u32 mul(u32 a,u64 z) {
15     return mul_t(a,trans(z));
16 }

```

5 Poly

5.1 多项式求逆

FILE poly/finv.cpp

LINES 97

USE ntt,多项式求逆

COST $O(n \log n)$

AUTHOR ysy

```

1 #include<iostream>
2 #include<cstdio>
3 #include<cstring>
4 #define int long long
5 using namespace std;
6 int read(){
7     int x=0,f=1;char ch=getchar();
8     while(ch<'0' || ch>'9'){if(ch=='-')f=-1;ch=getchar();}
9     while(ch>='0'&&ch<='9'){x=x*10+ch-'0';ch=getchar();}
10    return x*f;
11 }
12 const int N=4000010,mod=998244353,g=3;
13 int ksm(int p,int k){
14     int x=1;
15     while(k){
16         if(k&1)x=x*p%mod;
17         p=p*p%mod;
18         k>>=1;
19     }
20     return x;
21 }
22 int r[N];
23 void init(int lg){
24     for(int i=0;i<(1<<lg);i++){
25         r[i]=(r[i>>1]>>1)|((i&1)<<(lg-1));
26     }
27 }
28 void ntt(int *a,int n,bool f=0){
29     for(int i=0;i<n;i++){
30         if(i<r[i])swap(a[i],a[r[i]]);
31     }
32     for(int ln=2;ln<=n;ln<=1){
33         int x=ksm(g,(mod-1)/ln),mid=(ln>>1);
34         if(f)x=ksm(x,mod-2);
35         for(int i=0;i<n;i+=ln){
36             int y=1;
37             for(int j=0;j<mid;j++){
38                 int l=a[i+j],r=y*a[i+j+mid]%mod;
39                 a[i+j]=(l+r)%mod,a[i+j+mid]=(l-r+mod)%mod;
40                 y=y*x%mod;
41             }
42         }
43     }
44 }

```

```

43     }
44     if(f){
45         int iv=ksm(n,mod-2);
46         for(int i=0;i<n;i++){
47             a[i]=a[i]*iv%mod;
48         }
49     }
50 }
51 void mul(int *a,int *b,int n){
52     static int c[N],d[N];
53     memset(c,0,sizeof(c));
54     memset(d,0,sizeof(d));
55     int m=(n<<1);
56     for(int i=0;i<n;i++){
57         c[i]=a[i],d[i]=b[i];
58     }
59     ntt(c,m,0);ntt(d,m,0);
60     for(int i=0;i<m;i++){
61         a[i]=c[i]*d[i]%mod;
62     }
63     ntt(a,m,1);
64 }
65 void qinv(int n,int *g){
66     int m=1,lg=0;
67     static int f[N],h[N];
68     memcpy(f,g,sizeof(f));
69     memset(g,0,sizeof(g));
70     g[0]=ksm(f[0],mod-2);
71     while(m<=n){
72         m<<=1,lg++;
73         init(lg+1);
74         memcpy(h,g,sizeof(h));
75         memset(g,0,sizeof(g));
76         for(int i=0;i<m;i++){
77             g[i]=h[i]*2%mod;
78         }
79         mul(h,h,m);
80         mul(h,f,m);
81         for(int i=0;i<m;i++){
82             g[i]=(g[i]-h[i]+mod)%mod;
83         }
84     }
85 }
86 int n,a[N],b[N];
87 signed main(){
88     n=read()-1;
89     for(int i=0;i<=n;i++){
90         a[i]=read();
91     }
92     qinv(n,a);
93     for(int i=0;i<=n;i++){
94         cout<<a[i]<<" ";
95     }
96     return 0;
97 }

```

5.2 多项式乘法

FILE poly/ntt.cpp

LINES 73

USE ntt,未经任何优化

COST $O(n\log n)$

AUTHOR ysy

```

1 #include<iostream>
2 #include<cstdio>
3 #define int long long
4 using namespace std;
5 int read(){
6     int x=0,f=1;char ch=getchar();
7     while(ch<'0' || ch>'9'){if(ch=='-')f=-1;ch=getchar();}
8     while(ch>='0' && ch<='9'){x=x*10+ch-'0';ch=getchar();}
9     return x*f;
10 }
11 const int N=4000040,mod=998244353,g=3;
12 int n,m,a[N],b[N],r[N];
13 int ksm(int p,int k){
14     int x=1;

```

```

15     while(k){
16         if(k&1)x=x*p%mod;
17         p=p*p%mod;
18         k>>=1;
19     }
20     return x;
21 }
22 void init(int lg){
23     for(int i=1;i<(1<<lg);i++){
24         r[i]=((r[i>>1]>>1)|((i&1)<<(lg-1)));
25     }
26 }
27 void ntt(int *a,int lg,int fl=0){
28     int n=(1<<lg);
29     for(int i=0;i<n;i++){
30         if(i<r[i])swap(a[i],a[r[i]]);
31     }
32     for(int ln=2;ln<=n;ln<=1){
33         int mid=(ln>>1),x=ksm(g,(mod-1)/ln);
34         if(fl)x=ksm(x,mod-2);
35         for(int i=0;i<n;i+=ln){
36             int y=1;
37             for(int j=0;j<mid;j++){
38                 int l=a[i+j],r=a[i+j+mid]*y%mod;
39                 a[i+j]=(l+r)%mod,a[i+j+mid]=(l-r+mod)%mod;
40                 y=y*x%mod;
41             }
42         }
43     }
44     if(fl){
45         int iv=ksm(n,mod-2);
46         for(int i=0;i<n;i++){
47             a[i]=a[i]*iv%mod;
48         }
49     }
50 }
51 void mul(){
52     int lg=__lg(n+m)+1;
53     init(lg);
54     ntt(a,lg);ntt(b,lg);
55     for(int i=0;i<(1<<lg);i++){
56         a[i]=a[i]*b[i]%mod;
57     }
58     ntt(a,lg,1);
59 }
60 signed main(){
61     n=read(),m=read();
62     for(int i=0;i<=n;i++){
63         a[i]=read();
64     }
65     for(int i=0;i<=m;i++){
66         b[i]=read();
67     }
68     mul();
69     for(int i=0;i<=n+m;i++){
70         cout<<a[i]<<" ";
71     }
72     return 0;
73 }

```

6 String

6.1 后缀数组

FILE string/sa.cpp

LINES 34

USE 倍增算法求 sa, rk, height, 字符必须不超过 '~'

COST $O(n \log n)$

AUTHOR ppip

```

1  char s[N+5];
2  int id[N*2+5],sa[N+5],buc[N+5],h[N+5],rk[N+5];
3  int main() {
4      // read s
5      int n(strlen(s+1));
6      for (int i{1};i<=n;++i) ++buc[s[i]];
7      partial_sum(buc+1,buc+1+'~',buc+1);

```

```

8   for (int i{n};i>=1;--i) sa[buc[s[i]]--]=i;
9   int m{0};
10  for (int i{1};i<=n;++i) rk[sa[i]]=m+=(s[sa[i]]!=s[sa[i-1]]);
11  fill_n(buc+1,'~',0);
12  for (int w{1};m<n;w<=1) {
13      int cnt{0};
14      for (int i{0};i<w;++i) id[++cnt]=n-i;
15      for (int i{1};i<=n;++i) {
16          if (sa[i]>w) id[++cnt]=sa[i]-w;
17          ++buc[rk[i]];
18      }
19      partial_sum(buc+1,buc+1+m,buc+1);
20      for (int i{n};i>=1;--i) sa[buc[rk[id[i]]]--]=id[i];
21      fill_n(buc+1,m,0);
22      copy_n(rk+1,n,id+1);
23      m=0;
24      for (int i{1};i<=n;++i)
25          rk[sa[i]]=m+=(id[sa[i]]!=id[sa[i-1]]||id[sa[i]+w]!=id[sa[i-1]+w]);
26  }
27  // for (int i{1};i<=n;++i) cout<<sa[i]<<" ";
28  for (int i{1},k{0};i<=n;++i) {
29      k-=!!k;
30      while (s[i+k]==s[sa[rk[i]-1]+k]) ++k;
31      h[rk[i]]=k;
32  }
33  return 0;
34 }

```

6.2 后缀树

FILE string/sft.cpp

LINES 39

USE Ukkonen 在线构造隐式后缀树

COST build O(n)

AUTHOR ppip

```

1  constexpr int N(1e6),inf(1e7),RNG{26+26+10};
2  struct Suft {
3      int ch[N*2+5][RNG];
4      int st[N*2+5],len[N*2+5],link[N*2+5];
5      char s[N+5];
6      int now{1},rem{0},n{0},tot{1};
7      Suft() {len[0]=inf;}
8      int new_node(int l,int le) {
9          ++tot;st[tot]=l;len[tot]=le;
10         return tot;
11     }
12     void extend(int x) {
13         s[++n]=x;++rem;
14         for (int lst{1};rem;) {
15             while (rem>len[ch[now][s[n-rem+1]]])
16                 rem-=len[now=ch[now][s[n-rem+1]]];
17             int &v{ch[now][s[n-rem+1]],c{s[st[v]+rem-1]}};
18             if (!v||x==c) {
19                 link[lst]=now;lst=now;
20                 if (!v) v=new_node(n,inf);
21                 else break;
22             } else {
23                 int u{new_node(st[v],rem-1)};
24                 ch[u][c]=v;ch[u][x]=new_node(n,inf);
25                 st[v]+=rem-1;len[v]-=rem-1;
26                 link[lst]=v=u;lst=u;
27             }
28             if (now==1) --rem;
29             else now=link[now];
30         }
31     }
32     void search(int u,int dep=0) {
33         if (st[u]+len[u]-1>=n&&st[u]-dep!=n) cout<<st[u]-dep<<" ";
34         else dep+=len[u];
35         for (int i{0};i<RNG;++i)
36             if (ch[u][i]) search(ch[u][i],dep);
37     }
38 } T;
39 } T;

```


警钟长鸣

读题与建模

！ 多模拟样例

实现与边界

！ 任意范围修改、查询时，考虑复杂度会不会退化

思考误区

！ 发现新性质后，回头检查旧做法能否简化或是否已经不成立

！ DP 先想清楚有哪些可靠的状态和存储方式，再设计转移

提交之前

！ 检查多测清空

！ 检查 long long 和取模

！ TLE 时重新检查实际复杂度，尤其注意分治树等结构是否退化